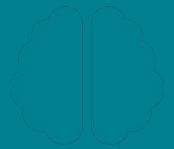

Single-Layer Networks

Activation Functions • Linear Separability • Perceptron Learning

Activation Functions

What are Activation Functions?

Mathematical functions that determine the output of a neuron
Transform input signals into output signals
Introduce non-linearity into the network
Enable networks to learn complex patterns

Key Properties

Range: The output range of the function
Differentiability: Essential for backpropagation
Computational Efficiency: Speed of computation during training

Sigmoid Activation Function

Mathematical Formula

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Properties

Range: (0, 1)
Shape: S-shaped curve
Output: Interpreted as probability

Derivative: σ'(x) = σ(x)(1 - σ(x))

Advantages

- Smooth gradient
- Output in (0,1) range
- Clear predictions

Disadvantages

- Vanishing gradient problem
- Not zero-centered
- Computationally expensive

Tanh Activation Function

Mathematical Formula

$$\tanh(x) = (e^{x} - e^{(-x)}) / (e^{x} + e^{(-x)})$$

Properties

Range: (-1, 1)
Shape: S-shaped, zero-centered
Relationship: Scaled sigmoid

Derivative: tanh'(x) = 1 - tanh²(x)

Advantages

- Zero-centered output
- Stronger gradients than sigmoid
- Better for hidden layers

Disadvantages

- Still suffers from vanishing gradient
- Computationally expensive (exp)
- Saturation at extreme values

ReLU Activation Function

Rectified Linear Unit

Mathematical Formula

$$\mathbf{ReLU(x) = max(0, x)}$$

Piecewise Definition:

$$f(x) = x \quad \text{if } x > 0$$
$$f(x) = 0 \quad \text{if } x \leq 0$$

Properties

Range: [0, ∞)
Non-saturating in positive region

Advantages

- Computationally efficient
- No vanishing gradient (x > 0)
- Sparse activation
- Biological plausibility

Disadvantages

- Dead ReLU problem
- Not zero-centered
- Unbounded output

Activation Functions Comparison

Function	Range	Advantages	Use Cases
Sigmoid	(0, 1)	Probability output, smooth gradient	Binary classification output layer
Tanh	(-1, 1)	Zero-centered, stronger gradients	Hidden layers in RNNs
ReLU	[0, ∞)	Fast, no vanishing gradient	Hidden layers in CNNs, DNNs

Key Takeaway

ReLU is the most commonly used activation function in modern deep learning due to its computational efficiency and ability to mitigate vanishing gradients.

Linear Separability

Definition

A dataset is linearly separable if there exists a hyperplane (line in 2D, plane in 3D) that can perfectly separate the data points of different classes.

Linearly Separable

- AND gate
- OR gate
- Binary classification with clear boundary

NOT Linearly Separable

- XOR gate
- Circular boundaries
- Complex decision boundaries

The XOR Problem

A Classic Example of Non-Linear Separability

XOR Truth Table

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Why XOR is Important

- No single line can separate the classes
- Requires non-linear decision boundary
- Single-layer perceptron cannot solve XOR
- Led to development of multi-layer networks

💡 **Single-layer networks can only solve linearly separable problems**

The Perceptron

The Simplest Neural Network

What is a Perceptron?

A single-layer neural network with binary output
Developed by Frank Rosenblatt in 1957
Foundation of modern neural networks

Mathematical Model

1. Weighted Sum: $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$

2. Activation: $y = 1 \text{ if } z \geq 0, \text{ else } y = 0$

Perceptron Learning Rule

Learning Algorithm

Weight Update Rule:

$$w_i(new) = w_i(old) + \eta \times (target - output) \times x_i$$

Where:

- η = learning rate (typically 0.01 to 0.5)
- target = desired output
- output = predicted output
- x_i = input feature

Perceptron Training Algorithm

1

Initialize

Set weights and bias to small random values or zero

2

Forward Pass

Calculate output: $y = f(w \cdot x + b)$

3

Calculate Error

$error = target - output$

4

Update Weights

$w = w + \eta \times error \times x$

5

Update Bias

$b = b + \eta \times error$

6

Repeat

Iterate until convergence or max epochs

Perceptron Convergence Theorem

Theorem Statement

If the training data is linearly separable, the perceptron learning algorithm is guaranteed to find a solution in a finite number of steps.

When It Works

- Data is linearly separable
- Guaranteed convergence
- Finds optimal decision boundary

Limitations

- Non-linearly separable data
- May never converge
- Cannot solve XOR problem

Summary

- Activation functions introduce non-linearity (Sigmoid, Tanh, ReLU)
- Linear separability determines single-layer network capability
- Perceptron learning rule updates weights based on error
- Convergence guaranteed only for linearly separable problems
- Multi-layer networks needed for complex problems (XOR)